UNTRM · INCID · FIZBAC · 2026-II

De un modelo en disco a un servicio

Una API que valida lo que recibe, una imagen que se construye igual en cualquier sitio y un primer CI que lo comprueba.

CASO: ONLINE RETAIL II | FASTAPI | DOCKER | GITHUB ACTIONS

Lo que construimos hoy

5 endpoints
1 imagen · 1 CI

El mismo modelo de S1, ahora al alcance de cualquier programa que sepa hablar HTTP, y con pruebas que demuestran que no cambia su respuesta por el camino.

AGENDA

Cuatro horas, cuatro bloques



0:00 · Revisión (30 min)

Entorno, pipeline y pruebas de S1 en verde. Comprobar que Docker responde en tu Codespace.

0:30 · Teoría (60 min)

Contrato de la API, errores y registros (45 min) y contenedor + primer CI (15 min).

1:30 · Práctica (135 min)

Arrancar la API, romperla a propósito, comparar predicciones, construir la imagen y ver el CI. Después, el laboratorio **exponer el servicio**.

3:45 · Entrega (15 min)

Se explica la tarea y la rúbrica, se resuelven dudas y se avanza a S3.

REVISIÓN · 30 MIN

¿Está listo el punto de partida?

```
conda activate env-ia-mlops
cd 04-Proyecto-Base
python -m ia_mlops.pipeline
python -m pytest -q -m "not lento"
```

> Esperado: `127 passed, 4 deselected`. El pipeline deja `models/modelo_mlflow/`: sin él, la API no tiene qué servir.

En tu Codespace

```
docker version
```

Deben aparecer **Client** y **Server**. Docker no se instala en tu equipo.

Si algo falla

- **Sin Codespace**: en pareja. El CI construirá la imagen.
- **Tests rojos**: partimos de `develop`, no de `tarea-s1`.

QUÉ CAMBIA HOY

Un archivo no es un servicio

Lo que le falta a `modelo.joblib`

- Que **otro programa** lo pueda llamar (HTTP)
- Que se **compruebe lo que llega** antes de predecir
- Que dé **la misma probabilidad** que en entrenamiento
- Que los fallos digan **quién se equivocó**
- Que diga si está **vivo** y si está **listo**
- Que deje **trazas** sin exponer datos

1. **Modelo con firma**
   formato MLflow, dentro de `models/`
2. **API**
   FastAPI: contrato, errores, registros
3. **Imagen**
   Docker: se ejecuta igual en cualquier sitio
4. **CI**
   GitHub Actions: lo comprueba en cada cambio
5. **Comparar**
   paquete · joblib · MLflow · API · contenedor

QUÉ ENTRA Y QUÉ SALE

El contrato de `POST /predecir`

VARIABLE	TIPO	REGLA
`recencia_dias`	entero	0 a 1000
`frecuencia`	entero	≥ 1 (sin historia no hay variables)
`monto_total`	decimal	≥ 0
`ticket_medio`	decimal	= monto_total / frecuencia
`unidades_total`	entero	≥ 0
`productos_distintos`	entero	≥ 0
`antiguedad_dias`	entero	≥ recencia_dias
`frecuencia_90d`	entero	≤ frecuencia_180d ≤ frecuencia
`monto_90d`	decimal	≤ monto_total
`frecuencia_180d`	entero	(ver arriba)
`dias_medios_entre_compras`	decimal o null	null ⇔ una sola factura
`devoluciones`	entero	≥ 0
`es_reino_unido`	0 o 1	solo esos dos valores

Respuesta (200)

```
{
  "probabilidad": 0.8279,
  "modelo": "regresion_logistica",
  "huella_datos": "35a94deb4e04ab4a",
  "advertencias": []
}
```

Un campo que **no está** en el contrato (como `y`) se rechaza con 422: no se ignora en silencio.

La firma es la fuente de verdad

```
firma = mlflow.models.infer_signature(muestra,
         modelo.predict_proba(muestra)[:, 1])
mlflow.sklearn.save_model(modelo, destino,
    signature=firma, input_example=muestra.head(3),
    serialization_format='cloudpickle')
```

Lo que declara la firma (real)

NAME	TYPE	REQUIRED
recencia_dias	long	true
monto_total	double	true
dias_medios_entre_compras	double	**false**
…10 más		

Tres guardias

- **Cada petición**: el esquema Pydantic valida tipos, rangos y coherencia.
- **Al arrancar**: firma = metadatos y esquema = firma. Si no, `/listo` responde 503.
- **En el CI**: pruebas de nombres, orden, tipos y obligatoriedad.

Reentrenas con una variable nueva y no tocas `esquemas.py`: la API **no queda lista** y el 503 nombra la variable que falta. Mejor eso que un 500 en cada predicción.

RECORRIDO

Seis estaciones, un solo viaje



3. Validación del contrato

ESQUEMAS.PY · CLIENTE

Pydantic comprueba tipos, rangos, campos desconocidos y la coherencia entre variables. El modelo todavía no se ha llamado.

> **Si algo va mal:** Si falla: 422 y se explica qué campo y por qué.

PREDICE EL CÓDIGO ANTES DE PULSAR

Qué responde la API

- **Cliente correcto**
- `recencia_dias = -1`
- Enviar también y
- Falta frecuencia
- `es_reino_unido = 2`
- `frecuencia_90d = 9`
- 6 facturas y sin días medios
- `ticket_medio` de 5 000
- Una factura, sin días medios
- Servidor sin modelo

Cambio respecto al cliente de ejemplo

`ninguno`

200 Todo bien: la API predice.

```
{
  "probabilidad": 0.8278889191932133,
  "modelo": "regresion_logistica",
  "huella_datos": "35a94deb4e04ab4a",
  "advertencias": []
}
```

La probabilidad viene con la huella de los datos con que se entrenó el modelo.

UNA DECISIÓN DE DISEÑO

Imposible se rechaza; desconocido se avisa

422 · SE RECHAZA

- Valor **imposible**: `recencia_dias = -1`
- Campo **desconocido**: `y`, `cliente_id`
- Combinación **incoherente**: más facturas en 90 días que en total
- Falta una variable obligatoria

Casi seguro es un error de quien llama; hay que decírselo.

200 · SE AVISA

- Valor posible que el modelo **no vio** al entrenar
- Falta `dias_medios_entre_compras` y hay una sola factura: se imputa

```
"advertencias": ["ticket_medio=5000.0 esta fuera del rango visto
al entrenar [34.78, 1294.87]; la prediccion no tiene respaldo
ahi."]
```

Rechazarlo todo dejaría sin respuesta a un cliente real; callarlo escondería que el modelo **extrapola**. Las advertencias quedan en la respuesta, para quien decide.

QUIÉN SE EQUIVOCÓ

200, 422, 503 y 500

CÓDIGO	SIGNIFICA	QUIÉN ACTÚA	QUÉ CONTIENE
`200`	Todo bien (quizá con advertencias)	—	probabilidad, modelo, huella, advertencias
`422`	El cuerpo no cumple el contrato	**Quien llama**	qué campo falla y por qué
`503`	No hay modelo o su contrato está roto	**Quien opera**	detalle + id de la petición
`500`	Fallo interno inesperado	**Quien desarrolla**	solo «Error interno» + id

En un 500 **no se cuenta nada interno**: ni rutas ni trazas. El traceback va al registro, con el mismo `id` que ve quien llamó.

Si un 422 pasara por 500, quien llama pensaría que el servicio está roto, y no que su dato es imposible.

DOS PREGUNTAS DISTINTAS

¿Está vivo? ¿Está listo?

Con modelo | Falta el modelo

`GET /salud` **200**

```
{"estado": "ok"}
```

`GET /listo` **200**

```
{"estado": "listo"}
```

`POST /predecir` **200**

```
{"probabilidad": 0.8279…, "advertencias": []}
```

Quién lo pregunta

- `/salud` · ¿reinicio el proceso? Si no responde, sí.
- `/listo` · ¿le envío tráfico? Si da 503, todavía no.

Kubernetes los llama sonda de vida (*liveness*) y de disponibilidad (*readiness*). El `HEALTHCHECK` del contenedor usa `/salud`.

OBSERVABILIDAD MÍNIMA

Una línea JSON por petición

```
{"severity": "INFO", "message": "modelo cargado", "evento": "modelo_cargado",
"huella_datos": "35a94deb4e04ab4a"}
{"severity": "INFO", "message": "POST /predecir 200", "evento": "peticion",
"id": "8879dadc7407", "metodo": "POST", "ruta": "/predecir", "codigo": 200,
"ms": 15.9}
{"severity": "INFO", "message": "POST /predecir 422", "evento": "peticion",
"id": "13c6ee4a5670", "metodo": "POST", "ruta": "/predecir", "codigo": 422,
"ms": 0.9}
```

Cabecera de la respuesta del 200: `x-request-id: 8879dadc7407`. Mismo id.

Qué se decide

- **Stdout**, no archivos: lo recoge quien ejecuta el contenedor.
- `severity` y `message`: los campos que Cloud Logging reconoce.
- **Nunca** los datos del cliente.

Defecto real de la solución de referencia: con `TestClient` las líneas aparecían; con `uvicorn` de verdad, ninguna. La prueba ahora arranca uvicorn como el contenedor.

COMPARAR PREDICCIONES

Mismo modelo, misma respuesta

```
200 filas reales de la particion test · tolerancia 1e-09
camino                          filas    diferencia maxima  resultado
paquete (referencia)              200            0.000e+00  coincide
joblib                            200            0.000e+00  coincide
MLflow                            200            0.000e+00  coincide
API http://127.0.0.1:8098         200            0.000e+00  coincide
Todos los caminos dan la misma probabilidad.
```

```
python -m ia_mlops.modeling.comparar --url
http://127.0.0.1:8080
```

Código de salida 0 si coinciden, 1 si difieren, 2 si no pudo comparar.

El cliente de ejemplo

Mi equipo (Linux)	0.8278889191932133
Contenedor del CI	0.8278889191932133

Entrenados en máquinas distintas con las mismas versiones de librerías.

Si difieren, **busca qué cambia**: versión de scikit-learn, orden de columnas, preproceso. Mejor antes de desplegar.

CONTENEDORES

Del Dockerfile al servicio en ejecución

1. **Dockerfile** — la receta, en texto
2. **`docker build`** — ejecuta la receta
3. **Imagen** — solo lectura, por capas
4. **`docker run`** — -p 8080:8080
5. **Contenedor** — un proceso aislado

Por qué

- Se ejecuta **igual** en tu Codespace, en el CI y, en S3, en Cloud Run.
- Lleva dentro sus **versiones exactas**.

Dónde se usa

- **Codespaces**: interactivo, para construir y probar.
- **GitHub Actions**: en cada cambio, sin tocar tu equipo.

> La imagen **no entrena**: sirve el modelo que dejó `make pipeline`. Entrenar y servir son pasos distintos.

RECETA

Cada línea tiene una razón

```
FROM python:3.11-slim

ENV PYTHONUNBUFFERED=1 … CURSO_MODELOS=/app/models

WORKDIR /app

COPY requirements-api.lock .
RUN pip install -r requirements-api.lock

COPY ia_mlops/ ia_mlops/
COPY models/modelo_mlflow/ …
COPY models/metadatos.json …

RUN useradd --uid 10001 …
USER 10001

EXPOSE 8080
HEALTHCHECK … /salud

CMD ["uvicorn", "ia_mlops.api.app:app",
     "--host", "0.0.0.0", "--port", "8080"]
```

4. Dependencias primero

53 paquetes con versión exacta. Esta capa solo se reconstruye si cambia el archivo de versiones: una línea de código nueva no reinstala todo.

> Orden de las capas = rapidez de las siguientes construcciones.

MÍNIMO PARA SERVIR

Solo lo necesario

ENTRA

- `ia_mlops/` (el paquete)
- `models/modelo_mlflow/` + `metadatos.json`
- **53 paquetes** con versión exacta (`requirements-api.lock`)
- `mlflow-skinny`: lee modelos y firmas

SE QUEDA FUERA

- Datos y la base SQLite
- Cuadernos, pruebas, `.git`
- Secretos habituales: `.env`, `*.key`
- `mlflow` completo, `modelo.joblib`

Imagen del CI (ejecución real)

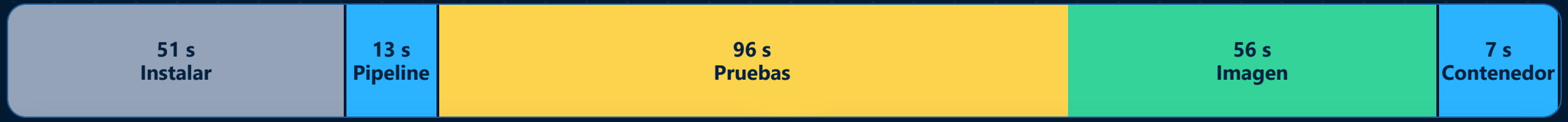
826 MB

en disco · 183 MB de «contenido» (docker image ls)

Ojo con `.dockerignore`: `*.pyc` solo vale en la raíz. Para cualquier carpeta se escribe `**/*.pyc`.

GITHUB ACTIONS · CI.YML

Qué hace en cada push



Construir la imagen

56 S · EJECUCIÓN REAL

docker build con deploy/docker/Dockerfile y docker image ls (826 MB en disco). Falla si falta models/ o si una línea del Dockerfile no se puede ejecutar.

Pasos medidos: 223 s. El trabajo completo tardó 3 min 54 s con la preparación del ejecutor.

Qué no hace (todavía)

No publica la imagen, no despliega, no usa secretos. Permisos: `contents: read`. Eso es S4.

Disparadores: `push` y `pull_request` a `develop` y `main`, y ejecución manual.

PRÁCTICA GUIADA · 135 MIN

De la API al contenedor



1 · Arrancar

25 MIN · DESDE EL MIN 10

make api, abrir /docs, probar /salud, /listo y /modelo, predecir con el cliente de ejemplo desde /docs y con curl, y un lote de dos.

Total de la práctica: 135 min.

Guía completa: `01-Sesiones/S2-API-y-Docker/practica/guia-laboratorio.md`

PRÁCTICA · BLOQUE 6 · LABORATORIO

Exponer el servicio

1. **Todo en mi ordenador**
   dos procesos en 127.0.0.1: funciona, pero no sale de ahí
2. **La API en su imagen**
   lo del bloque 4: igual en cualquier equipo con Docker
3. **La interfaz en su imagen**
   ids → variables (SQL de FS) → API · sin modelo ni datos
4. **Dos contenedores que se hablan**
   red · la API sin -p · datos con -v …:ro
5. **Lo mismo con Compose**
   un archivo: deploy/docker/compose.yaml
6. **¿Ya es producción?**
   lo que falta y en qué sesión se ve — S3 · S4 · S5

La pregunta

La API pide 13 variables; quien hace la campaña solo tiene **ids de cliente**. ¿Dónde llama y qué recibe? Una **interfaz** con probabilidad, tramo y recomendación.

Antes de empezar

Pipeline ejecutado y la imagen `retencion-api:local` del bloque 4, en tu Codespace.

Presentación del laboratorio: **Del ordenador al servicio** · 36 diapositivas · salidas y capturas reales

Abrir el laboratorio →

En texto: `docs/del-ordenador-al-servicio.md`

ENTREGA

Tu API, tu contenedor, tus comparaciones

Pasos

- Una **API** sobre el modelo de tu caso, con el contrato tomado de la **firma**.
- Errores 422, 503 y 500 **distinguibles** y probados.
- `Dockerfile`, `.dockerignore` y versiones exactas.
- Un **workflow** que construye y prueba la imagen.
- **Compara** paquete, joblib, MLflow, API y contenedor.

Rúbrica



- **20 %** Reproducibilidad
- **40 %** Técnica (contrato, errores, imagen)
- **40 %** Análisis y decisiones

ANTES DE EMPEZAR

Errores que ya vimos

COPY falla al construir

No existe `models/modelo_mlflow/`: ejecuta antes `make pipeline`.

/listo da 503

Falta el modelo, o el esquema de la API no coincide con la firma. Lee el `detalle`.

El navegador no llega al contenedor

Falta `-p 8080:8080`, o uvicorn escucha en 127.0.0.1 en vez de 0.0.0.0.

Imagen enorme o con datos

Falta `.dockerignore`, o un patrón sin `**/`.

422 por un campo que «sobra»

Es el contrato: `y` no puede viajar. Quítalo.

Windows: `curl`

En PowerShell usa `curl.exe`; en CMD sirve `curl`.

CIERRE

Qué te llevas y qué viene

Hoy

- Un contrato que sale de la **firma** del modelo
- Errores que dicen **quién** se equivocó
- Una **imagen** mínima y un **CI** que la prueba
- Predicciones **idénticas** por todos los caminos

S3

- De local a **GCP**: inventario y plan
- Datos a Cloud Storage y BigQuery
- Secret Manager y Artifact Registry
- Esta imagen, en **Cloud Run**

Pendiente para ti: **la tarea** y anotar tus incidencias con Docker o con el CI.